

クラウドLLMとローカルLLMの間にプライバシー境界を実装するLLM呼び出しフレームワークの比較調査

エグゼクティブサマリー

本調査では、クラウドLLMとローカルLLMを併用するハイブリッド運用と、明示的なプライバシー／データ境界制御の両方を重視して、主要候補を六つに絞って比較しました。結論から言うと、最も“境界”を明示的に扱っているのは **NBAccess** で、ノートブックやセルの機密性を `AccessLevel` や `CloudPublishable -> False` で宣言し、その結果に基づいて **機密データはローカルLLMにしか送らない** という設計が前面に出ています。一方、企業向けの実運用で最も完成度が高いのは **Portkey** と **Kong AI Gateway** で、前者はハイブリッド／エアギャップ配置、PII/PHI対応のガードレール、VPC内データプレーン、監査ログ、RBAC を組み合わせやすく、後者は自己ホスト前提のPIIサニタイズ、セマンティックルーティング、詳細なAI監査ログが強みです。高スループットと低コスト最適化なら **LiteLLM** が最も実務的で、公式ベンチマークでも 1k RPS 時の P95 8ms などの具体値が公開されています。**Haystack** と **LangChain/LangGraph** は、プライバシー境界を“フレームワークが強制する”というより、PIIマスキング・ルータ・トレーシング制御を組み合わせるアプリ層で設計するタイプです。 ①

用途別に見ると、最高機密・社内閉域・厳格なデータ境界では、ノートブック中心なら **NBAccess**、企業共通ゲートウェイなら **Kong AI Gateway** か **Portkey Hybrid** が第一候補です。低コストでは、ローカルモデルや OpenAI-compatible な self-hosted endpoint を前段でさばきつつ、必要時のみ高価なクラウドに切り替えられる **LiteLLM** が最も扱いやすいです。高スループットでは、公開ベンチマークがある **LiteLLM** が優勢で、**Portkey** は自社公開の自己申告値として低オーバーヘッドを掲げるものの、第三者比較ベンチマークは確認できませんでした。可観測性・監査では、**Kong**、**Portkey**、**LangSmith** 系が強く、**Haystack** と **LangChain OSS** 単体、**NBAccess** はこの点で周辺基盤への依存が大きいです。 ②

重要な注意点として、差分プライバシー differential privacy は、今回比較した“LLM呼び出しフレームワーク”の現行公式文書では第一級機能としてはほぼ見当たりませんでした。DP は現在も主に 学習・微調整側の研究テーマであり、呼び出しゲートウェイやオーケストレータでの境界制御は実際には、データの局所処理、PII 検知・マスキング、トレース抑止、アクセス制御、ルーティング規則、ログ最小化で構成されています。したがって実務では、「DP があるか」よりも「どの入力、どの条件で、どの観測系に残り、どの provider に送られるかを説明できるか」の方が重要です。 ③

調査範囲と評価軸

このレポートでいう「フレームワーク」は、狭義のSDKだけでなく、LLM呼び出し経路に介在してルーティング・ガードレール・監査を行う **AI Gateway** も含めています。ユーザー要件の「NBAccess のようなプライバシー境界」を満たすため、選定基準は次の二つを必須にしました。第一に、クラウドLLMとローカルLLMの双方を扱えること。第二に、明示的なプライバシー／データ境界制御がドキュメント化されていること、または少なくとも PII 検知・ローカル優先・トレース抑止・アクセス制御などを組み合わせて境界を記述できることです。**Haystack** や **LangChain/LangGraph** のようなアプリ層フレームワークは後者を「構成可能な実装」で満たし、**Portkey**・**Kong**・**LiteLLM** は主に「ゲートウェイ／ポリシー層」で満たします。 ④

評価軸は、ユーザー指定の観点を整理し、実測可能なものと、設計レビューで評価すべきものに分けました。性能系は、スループット、追加レイテンシ、キャッシュ有無、フェイルオーバー時の復旧動作。コスト系は、リクエスト単価、ゲートウェイ運用費、観測系ストレージ費、ローカルGPU費。プライバシー系は、

オンデバイス処理可否、PII 検知・匿名化、トレース／ログに残るデータの最小化、暗号化、RBAC、監査ログ、そして“fail-open / fail-closed”の既定挙動です。**開発者体験**では、統一API、ポリシー定義の容易さ、既存SDK統合、自己ホスト容易性、OpenAI互換性を重視しました。Kong、LiteLLM、Portkey は AI Gateway としてこの評価軸に乗せやすく、Haystack と LangChain は“アプリでどこまでガードを埋め込めるか”を見る必要があります。 ⁵

評価メトリクスは、実査しやすい形に落とすと次のようになります。

観点	推奨メトリクス	測定方法の要点
スループット	requests/sec、tokens/sec	同一プロンプト長・同一モデルで、ゲートウェイ有無の差分を比較
レイテンシ	p50 / p95 / p99 追加遅延	provider 応答時間を除いた middleware / gateway overhead を分離測定
コスト	1 request あたり総コスト	APIトークン課金 + キャッシュ効果 + ゲートウェイ運用費 + GPU/VM費
プライバシー漏えいリスク	漏えい面積スコア	送信先数、外部トレース有無、PIIマスキング、ログ既定値、暗号化を合成評価
ポリシー記述性	ルール記述ステップ数、変更反映時間	「PII検知→ローカルへ」「公開可能→クラウドへ」を何行・何設定で書けるか
フェイルオーバー	成功率、誤送信率、タイムアウト時既定挙動	障害注入で cloud へ誤送信しないか、guard timeout で fail-open にならないか確認

プライバシー漏えいリスクを重視する理由は、生成AIの文脈で **PII 漏えいや訓練データ起因の再生成リスク** が実証研究で繰り返し指摘されているためです。実運用では「モデル自体の漏えいリスク」に加え、「**アプリが不要に外部 provider や外部 observability にデータを出すリスク**」を別管理する必要があります。 ⁶

主要フレームワーク比較

以下の表では、今回の比較対象について、ユーザー要件に直接関係する項目だけを圧縮して示します。

フレームワーク	類型	ハイブリッド対応	プライバシー境界モデル	ルーティング	観測・監査
NBAccess	ノートブック拡張	OpenAI / Anthropic と LM Studio を同一インターフェースで管理	AccessLevel、CloudPublishable -> False、 providerごとの最大アクセスレベル、機密セルはローカルのみ	ルールベース、機密度ベース、fallback model 選択	監査ログや統合 observability は限定的。APIキーは SystemCredential 統一管理

フレームワーク	類型	ハイブリッド対応	プライバシー境界モデル	ルーティング	観測・監査
LiteLLM	OSS AI Gateway / SDK	100+ LLM、vLLM、OpenAI-compatible、ローカル proxy を統一APIで扱える	ログ抑止、guardrails、virtual keys、model access、secret managers。境界は gateway policy と config で実装	simple-shuffle / least-busy / usage-based / latency-based / cost-based、fallback / retry	UI logs、callback、OpenTelemetry、監査ログ、SSO/RBAC は Enterprise
Portkey	OSS + 商用 AI Gateway	private LLM deployments、Hybrid/Airgapped、AWS/GCP/Azure private cloud	Data Plane を顧客 VPC内、Control Plane 分離、PII/PHI guardrails、RBAC、監査ログ、BYOK、E2E encryption	conditional routing、fallback、retry、load balancing、provider optimization	Logs / Traces / Alerts / FinOps、監査ログ
Kong AI Gateway	企業向け AI Gateway	Universal LLM API で OpenAI, Anthropic, Gemini, Bedrock, Azure AI など。自己ホスト前提で private 配置しやすい	allow/deny lists、PII sanitizer、private/self-hosted、plugin ベースの semantic security	weighted / consistent-hashing / least-connections / lowest-usage / lowest-latency / semantic / priority	詳細な AI audit logs、PII sanitizer logs、semantic cache logs、MCP logs
Haystack	OSS アプリオーケストレータ	OpenAI など cloud と Ollama / HuggingFace Local / Llama.cpp / vLLM を同一 pipeline で組める	Presidio で PII anonymization、content tracing は既定で無効、境界は pipeline 設計次第	ConditionalRouter、FallbackChatGenerator、metadata / language routers	tracing は OpenTelemetry / Datadog / Langfuse 等。content tracing は既定で off
LangChain / LangGraph	OSS アプリフレームワーク	OpenAI, Anthropic, Google, Ollama など。LangGraph は local server / Docker data plane を持つ	PIIMiddleware、guardrails、Human-in-the-loop、OTEL redaction、checkpoints の AES 暗号化	middleware による rules、fallbacks、retries。境界は実装可能だが既定で強制されない	LangSmith observability、audit logs、OTEL 経路 redaction、条件付き tracing

性能と価格公開度を切り出すと、**官方ドキュメントで具体的な数値を最も出しているのは LiteLLM**です。LiteLLM は公式ベンチマークで **1k RPS 時の P95 8ms**、4 インスタンス構成でより高い集約スループットを掲載しています。Portkey は GitHub README で **<1ms latency** や **10B tokens/day** を掲げていますが、詳細な再現条件付き比較は公開ベンチマークほど整っていません。Kong、Haystack、LangChain/LangGraph、NBAccess は、今回レビューした公式資料の範囲では中立的な比較ベンチマークが不足していました。そのため、この領域では“**未公表**”をそのまま未公表として扱うのが妥当です。 13

分析と推奨

NBAccess の最大の価値は、プライバシー境界が“概念”ではなく **データ属性として明示化されている点**です。`CloudPublishable -> False` と notebook-required access level、provider ごとの最大アクセスレベルを用いて、「このノートブック／このセルはクラウドへ出してよいか」をまず判断してから LLM を呼ぶ構造になっており、ユーザーが最初に挙げた要件にもっとも近いです。さらに、fallback model も access level に従って選び分けられるため、**可用性のためのフェイルオーバーが、境界違反を起こさないよう設計されているのが重要**です。ただし、適用対象が事実上 Wolfram Notebook であり、一般的な REST / microservice ベースの LLM gateway ではないため、**全社共通の LLM 呼び出し基盤**としては守備範囲が狭いという限界があります。 14

LiteLLM は、厳密な意味での“プライバシー境界フレームワーク”ではなく、**境界を構成可能にする AI Gateway** です。強みは三つあります。第一に、**クラウド provider とローカル／OpenAI-compatible endpoint をまとめて扱えること**。第二に、**cost-based / latency-based / least-busy / usage-based** など、運用現場で効くルーティング戦略が揃っていること。第三に、**guardrails、virtual keys、budgets、secret managers、message logging off by default** といった運用機能が現実的に充実していることです。したがって LiteLLM は、「**PII 検知や model access 制約を前段でかけ、基本はローカルや安価モデルに流し、必要時のみ高性能 cloud に昇格**」という低コスト・高スループット設計に向きます。一方で、NBAccess のような **データフローに内在した taint／access 判定** を自動でしてくれるわけではないため、境界の厳格さは config と周辺ポリシー整備に依存します。 15

Portkey は、今回比較した中で“**企業ガバナンスとハイブリッド配置**”のバランスが最もよい候補です。顧客 VPC に Data Plane を置き、Control Plane は分離しつつ、**敏感な LLM データと AI traffic を自社環境に留める設計**が明示されています。さらに、**PII/PHI redaction、監査ログ、組織単位メタデータ強制、BYOK、Hybrid / Airgapped / SaaS の複数配置**が揃っているため、規制産業や大企業の“**共通 AI 接続レイヤ**”に向きます。弱点は、上位機能の多くが有償プラン寄りで、また Hybrid でも Control Plane は Portkey 側に残るため、**完全な自社完結を求める組織では Airgapped か別候補の検討が必要**になることです。 9

Kong AI Gateway は、自己ホストで強い統制を持ちたい組織に非常によく合います。特に、**going privately and self-hosted、PII sanitization、semantic routing、priority failover**、そして **AI plugin ごとの詳細 audit log schema** が揃っている点は優秀です。Kong の設計哲学は、アプリの中に privacy logic を埋め込むのではなく、“**すべての LLM traffic を connectivity layer で統制する**”ことにあります。このため、マルチチーム・マルチアプリで共通ポリシーを敷く用途では非常に強い反面、意味論や業務ルールに深く結びつく privacy policy は、結局アプリや外部 guardrail service と連携する必要があります。また、AI plugin の多くがライセンス必須である点はコスト計画に影響します。 10

Haystack は、研究開発から実運用へ移るときに“**境界をコードで丁寧に組みたい**”チームに向きます。`ConditionalRouter` と `FallbackChatGenerator` によって、**PII sanitizer → route to local / sanitized cloud** のような DAG を比較的きれいに書けますし、OpenTelemetry や Langfuse などの tracing もあります。とくに評価できるのは、**content tracing が既定で無効**であることです。これは observability で事故りやすい「プロンプト本文を外部 trace backend に送ってしまう」問題を抑える意味で実務的です。ただし、Haystack 自身がネットワーク境界や組織RBACを強制するわけではないので、**セキュリティ境界の“実効**

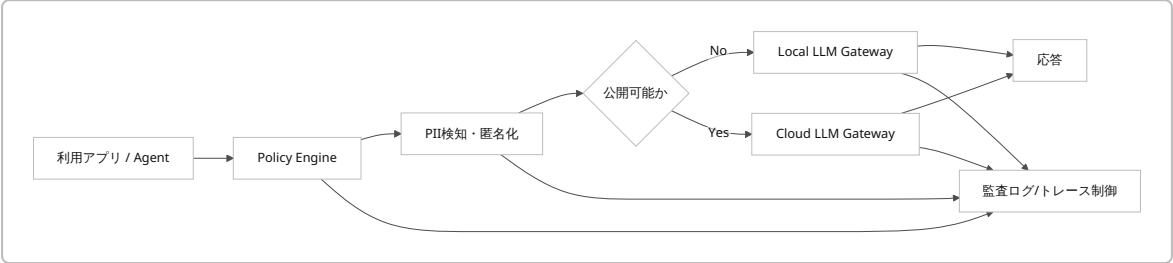
性”は周辺の API gateway、Kubernetes network policy、secret 管理、SIEM 連携で補う前提になります。 16

LangChain / LangGraph は、フレームワークとしては最も柔軟ですが、プライバシー境界という観点では「実装できること」と「既定で安全であること」は別です。PIIMiddleware、guardrails、Human-in-the-loop、OTEL collector 経由の redaction、langgraph up の AES checkpoint encryption、LangSmith の監査ログと observability といった部品は非常に豊富です。しかし、それらはミドルウェアや運用ポリシーとして設置する必要があるため、開発チームが provider を直接呼べてしまう構成だと境界は迂回されます。したがって LangChain/LangGraph は、アプリ内ルーティングや agent 実装には強いが、全社境界の enforce には単体では弱いという評価になります。全社標準にしたいなら、前段に LiteLLM / Portkey / Kong などの gateway を置き、その後ろで LangGraph を動かすのが安全です。 17

用途別の推奨をまとめると、高プライバシーは「Notebook 中心なら NBAccess」「全社ゲートウェイなら Kong AI Gateway または Portkey Hybrid」。低コストは、ローカル推論と cost-based routing を組み込みやすい LiteLLM が第一候補で、Haystack もローカル構成では有力です。高スループットは、現時点で公開ベンチマークがある LiteLLM を第一候補、Portkey を第二候補、Kong は大規模プラットフォーム統制寄りとするのが妥当です。開発者体験と柔軟性だけなら LangChain/LangGraph が最も高いですが、境界 enforcement を gateway 側に寄せる設計とセットで採用すべきです。 18

アーキテクチャパターンと実装チェックリスト

厳格なプライバシー境界を実装するときは、アプリから直接 provider を呼ばせず、policy engine と gateway を必ず一段挟むのが基本です。NBAccess は notebook 内部でこれを行っていますし、Kong・Portkey・LiteLLM はネットワーク境界として実装できます。Haystack や LangGraph を使う場合も、「アプリ内分岐」だけで済ませず、最終送信先を gateway に一本化すると監査可能性が大きく上がります。 19



上図の設計で重要なのは、PII検知と routing を別レイヤにしないことです。PII を検知しても、その結果が routing に確実に反映されなければ、ガードは“通知”で終わります。NBAccess は access level と provider access level を直接結び付けるのが強く、Kong は PII sanitizer と routing / plugin logs、Portkey は guardrails と hybrid dataplane、LiteLLM は model access / guardrails / routing strategies を組み合わせる形です。Haystack と LangGraph はこれを コードレベルで作れますが、作れることと、全チームで破れないことは違うため、本番では gateway を前段に置くべきです。 20

コストについては、現実には framework 利用率よりも モデル従量課金と運用コスト が支配的です。以下のよう
に考えると設計判断がしやすくなります。

パターン	典型構成	コスト式の考え方	向くフレームワーク
完全ローカル	すべて Ollama / vLLM / LM Studio	GPU/CPU費 + 電力/運用 + observability保管費	NBAccess, LiteLLM, Haystack, LangGraph

パターン	典型構成	コスト式の考え方	向くフレームワーク
サニタイズ 後クラウド	PIIを除去して cheap cloud へ、難問のみ premium cloud	$N \times \text{cheap API} + \beta \times \text{premium API} + \text{gateway 運用費}$	LiteLLM, Portkey, Kong
ローカル優 先ハイブ リッド	既定 local、失敗時または公 開可能時のみ cloud	$\alpha \times \text{local infra} + (1-\alpha) \times \text{cloud API} - \text{cache savings}$	NBAccess, LiteLLM, Portkey, Haystack
全社共通境 界	全アプリが同一 gateway を 使う	API従量 + gateway license / infra + SIEM / trace 費	Portkey, Kong, LiteLLM

短い実装チェックリストとしては、次の順序が実務的です。

- **送信先を一本化する。** すべての LLM 呼び出しを gateway 経由にし、provider 直呼びを禁止する。Portkey、Kong、LiteLLM はこの用途に向きます。²¹
- **データ分類を先に決める。** `public / internal / confidential / regulated` のような access level を定義し、NBAccess のように routing に直結させる。²²
- **ログ既定値を最小化する。** LiteLLM は request/response content を既定で保存せず、Haystack は content tracing を既定で無効にしています。LangChain/LangGraph も tracing を無条件で有効にしない方が安全です。²³
- **fail-open を明示的に潰す。** guardrail timeout や provider failure 時に cloud 側へ誤送信しないか、priority fallback が privacy policy を破らないかを障害注入で確認する。Kong と Haystack は failover の機構が明示され、LiteLLM は fallback 周りが豊富です。²⁴
- **可観測性を二段にする。** リクエスト本文を持たない運用メトリクスと、必要時のみ redacted traces を分離する。LangChain は OTEL collector で redaction を前段適用できます。²⁵
- **コンプライアンスの“実体”を確認する。** SOC 2 や HIPAA と書かれていても、どのプランで、どの deployment mode に適用されるかまで契約前に確認する。Portkey と LangSmith はこの点が明示的です。²⁶

明示的な前提と未確定事項

本レポートには、比較のために明示しておくべき前提があります。まず、“**framework**”を **SDK / orchestration library だけでなく AI Gateway まで含めている**ことです。ユーザーが求めた「プライバシー境界」はアプリ層だけではなく、実務上はむしろ gateway 層に現れるため、この広い定義を採用しました。²⁷

次に、**実証ベンチマークは公開されているものだけを採用し、なければ未公表と扱った**ことです。LiteLLM には具体的な数字がありますが、Portkey・Kong・Haystack・LangChain・NBAccess については、今回確認した公式文書だけでは横比較可能な中立ベンチマークが不足していました。このため、「高スループット推奨」は公開数値の有無も重く評価しています。²⁸

また、**差分プライバシーは current-generation の呼び出しフレームワークにはほぼ組み込まれていない**という整理も、公式 docs の記載と研究文献の対照から得た分析的結論です。すなわち、DP は fine-tuning / training 側で研究が進んでいる一方、今回の比較対象で前面に出てくる privacy control は、**PII redaction、アクセス制御、ローカル処理、暗号化、監査ログ、トレース抑止**です。したがって、要件定義では DP の有無を絶対条件にするより、「どこに送るか」「何を残すか」「誤送信時にどう止まるか」を契約・実装・監査の単位に落とす方が現実的です。²⁹

最後に、LangChain / Haystack のようなアプリフレームワークは、“境界を実装できる”が“境界を強制する”とは限らない、という点を明示しておきます。これは欠点というより設計哲学の違いです。アプリ層の柔軟性が必要なら優秀ですが、**全社で境界を均一化したいなら gateway 併用が前提**になります。その意味で、最終的な推奨アーキテクチャは、**Portkey / Kong / LiteLLM のいずれかを前段境界に置き、NBAccess・Haystack・LangGraph をその背後のアプリ実装に使い分ける**形です。これが、ユーザー要件に対して最も再現性が高く、監査もしやすい構成です。 30

1 7 14 19 20 22 GitHub - transreal/NBAccess · GitHub

<https://github.com/transreal/NBAccess>

2 13 18 28 Benchmarks | liteLLM

<https://docs.litellm.ai/docs/benchmarks>

3 29 Fine-Tuning Large Language Models with User-Level ...

https://arxiv.org/abs/2407.07737?utm_source=chatgpt.com

4 11 Get Started | Haystack Documentation

<https://docs.haystack.deepset.ai/docs/get-started>

5 24 Load balancing with AI Proxy Advanced - Kong Gateway | Kong Docs

<https://developer.konghq.com/ai-gateway/load-balancing/>

6 Analyzing Leakage of Personally Identifiable Information in ...

https://gangw.cs.illinois.edu/class/cs562/papers/llm-leak-sp23.pdf?utm_source=chatgpt.com

8 15 27 LiteLLM - Getting Started | liteLLM

<https://docs.litellm.ai/>

9 26 Feature Comparison - Portkey Docs

<https://docs.portkey.ai/docs/product/product-feature-comparison>

10 GitHub - Kong/kong: The API and AI Gateway · GitHub

<https://docs.konghq.com/install/source>

12 LangChain overview - Docs by LangChain

<https://docs.langchain.com/oss/python/langchain/overview>

16 ConditionalRouter | Haystack Documentation

<https://docs.haystack.deepset.ai/docs/conditionalrouter>

17 30 Overview - Docs by LangChain

<https://docs.langchain.com/oss/python/langchain/middleware/overview>

21 AI Gateway - Portkey Docs

<https://docs.portkey.ai/docs/product/ai-gateway>

23 Getting Started with UI Logs | liteLLM

https://docs.litellm.ai/docs/proxy/ui_logs

25 Redact sensitive data with the OpenTelemetry Gateway architecture - Docs by LangChain

<https://docs.langchain.com/langsmith/otel-gateway-trace-redaction>